Teste prático Verx - Documentação de Arquitetura

Visão geral do design da solução, decisões arquiteturais (CQRS & Event-Driven) e Stack Tecnológica.

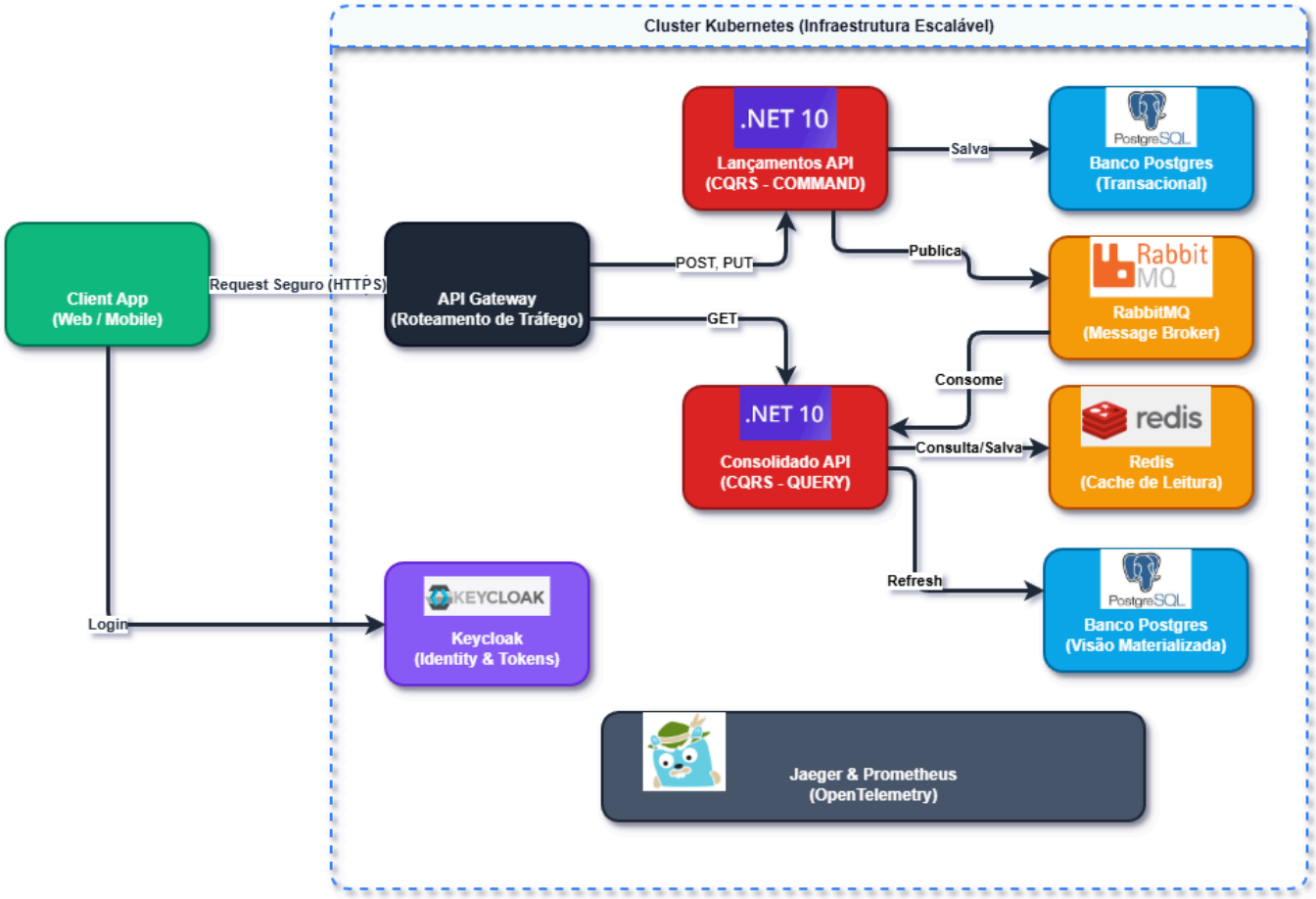
---

1. Visão Geral da Solução

O sistema foi construído visando **alta disponibilidade, escalabilidade e resiliência**. O domínio da aplicação consiste na gestão de lançamentos financeiros (crédito e débito) e na consolidação e exibição de um saldo diário. Para suportar um cenário onde o volume de leituras do saldo pode divergir amplamente do volume de inserções de caixa (por exemplo, sistemas muito intensos em leitura na virada do dia ou de grande volume de transações ao longo do dia), optou-se pela separação conceitual e física das responsabilidades.

2. Decisões Arquiteturais

A arquitetura principal repousa em dois pilares: **CQRS (Command-Query Responsibility Segregation)** a nível de microsserviços e **Event-Driven Architecture (Arquitetura orientada a eventos)**.



Microsserviço de Lançamentos (Escrita)

Neste módulo ocorre a ingestão dos dados. Ele representa o **Command** da nossa arquitetura. Suas funções são validar regras de negócio rígidas (se o valor do lançamento é positivo, autenticação, etc) e persistir o dado no banco de forma segura.

Ele não se preocupa em acumular saldos. O principal objetivo é processar a entrada de dados com throughput extremo.

Microsserviço de Consolidado (Leitura)

Este módulo representa a **Query**. Construído para ser extremamente rápido, é o serviço consultado pelo usuário final ou outros sistemas para resgatar o saldo. Para não impactar o banco de dados principal de escritas de Lançamento, ele mantém sua própria base de dados (projeção local) otimizada para a leitura do dia e emprega Caching Distribuído.

2.1 Por que 2 Microssegviços e Padrão CQRS?

- **Escalabilidade Independente:** Em dias de pico volumétrico (ex: Black Friday, pagamento de folhas), podemos escalar o banco de dados e os pods da API

(Containers) do 'Lançamentos' e 'Consolidado' de forma separada segundo a necessidade de Hardware.

- **Isolamento de Falhas:** Se o serviço ou banco de dados que consulta o saldo diário cair, o caixa continua realizando recebimentos e inserindo dados normalmente, mantendo a empresa operante.
- **Otimização de Modelagem:** A base de Lançamentos usa uma modelagem voltada para Inserção (Append-Only para transações), enquanto a Consolidado tem as tabelas desnormalizadas e apenas agregadas por dia, somadas ao disparo via Cache Redis. Leituras ocorrem em O(1) sem agredir consultas (JOINs, SUMs) custosas.

2.2 Comunicação Assíncrona e Event-Driven (RabbitMQ)

Para unir o mundo da Escrita com o da Leitura sem criar acoplamento temporal (onde um serviço derrubaria o outro através de HTTP Direto/REST se estivesse lento), foi adotado um modelo **Event-Driven (Mensageria)** usando o RabbitMQ e o core do MassTransit:

Sempre que o Serviço de Lançamentos persite uma operação (Ex: Débito de $50), ele emite um evento (`LancamentoCriadoEvent`) para um Message Broker interno. Imediatamente (trazendo o conceito de Consistência Eventual e Tempo Real), o serviço de Consolidado reage a essa mensagem lendo a fila, calculando o saldo localmente em seu banco próprio banco, invalidando os caches e preparando a vitrine de leituas novamente.

3. Stack Tecnológica Base

Para assegurar que as melhores práticas de mercado foram aplicadas e demonstrar domínio sobre stacks corporativos de ponta, o ecossistema abrange tecnologias de Alta Performance e Observabilidade nativa:

.NET 10 (C#) **PostgreSQL** **Redis Cache** **RabbitMQ** **MassTransit**

Keycloak (OAuth2 / JWT) **OpenTelemetry** **Jaeger** **Prometheus**

Docker + Docker Compose **SonarQube**

- **Segurança Centralizada (Keycloak):** A proteção de endpoints foi delegada ao Keycloak atuando como Identity Provider emitindo JWTs, simplificando a gerência de autenticação que requer Zero Trust Security e tirando do desenvolvedor a responsabilidade de gerenciar as senhas criptografadas.
- **Cache Alta Volagem (Redis):** No microserviço Consolidado, antes de efetuar consultas ao PostgreSQL, a aplicação verifica a existência de dados quentes no Redis. Isto tira drásticamente a carga do banco tradicional e entrega requisições em latências menores.
- **Observabilidade Unificada:** Toda a rastreabilidade da requisisão HTTP até as mensagens enfileiradas no RabbitMQ estão instrumentadas via *OpenTelemetry* com métricas de negócios expostas via prometheus e o tracking de logs no Jaeger.

4. Qualidade de Código & Clean Architecture

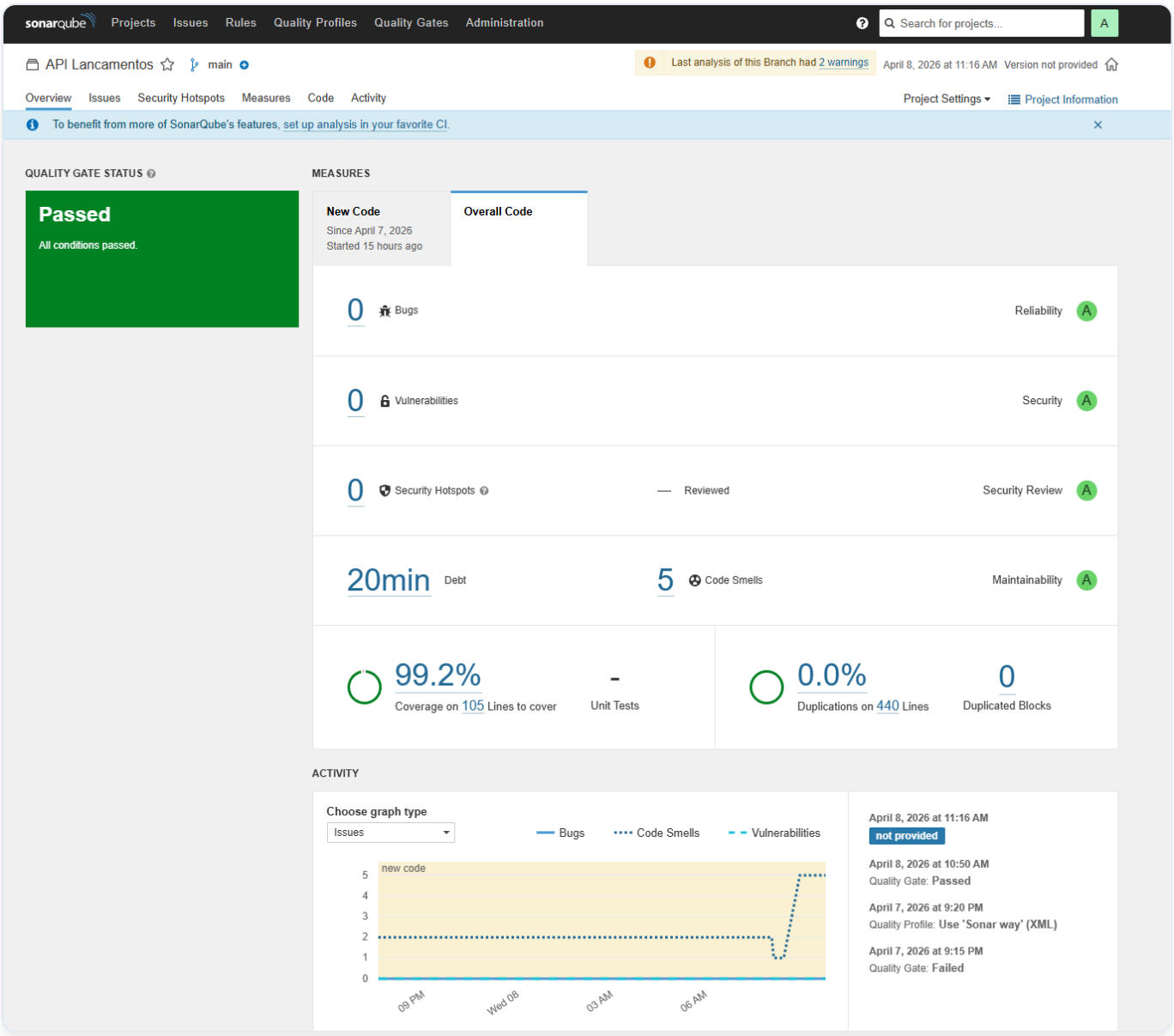
O conceito de Clean Code e Clean Architecture foi adotado de ponta a ponta no repositório.

Todas as entidades redundantes foram unificadas via camadas de `**`Domain e Contracts`**`. As regras de exceções, falhas em bancos e persistencia async assíncrona foram amplamente testadas utilizando `xUnit, Moq, FluentAssertions` com suporte de Mock HTTP e MessageBroker em cenários unitários e de Integração.

Resultados do Quality Gate no SonarQube

Comprovando as boas práticas e atendendo os requisitos do processo, os resultados das varreduras atestam **elevada Cobertura de Testes Unitários (>80%)**, ausência de duplicidades críticas (Duplicates reduzidas) e 0 Bugs críticos.

Dashboard - Microsserviço API Lançamentos



Dashboard - Microsserviço API Consolidado

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration

Search for projects... A

API Consolidado main

Last analysis of this Branch had 2 warnings April 8, 2026 at 11:16 AM Version not provided

Overview Issues Security Hotspots Measures Code Activity

Project Settings Project Information

To benefit from more of SonarQube's features, set up analysis in your favorite CI.

QUALITY GATE STATUS

Passed

All conditions passed.

MEASURES

New Code
Since April 7, 2026
Started 15 hours ago

Overall Code

0 Bugs — Reliability A

0 Vulnerabilities — Security A

0 Security Hotspots — Reviewed — Security Review A

0 Debt — 5 Code Smells — Maintainability A

84.5% Coverage on 146 Lines to cover — - Unit Tests

0.0% Duplications on 518 Lines — 0 Duplicated Blocks

ACTIVITY

Choose graph type: Issues

Bugs · Code Smells · Vulnerabilities

April 8, 2026 at 11:16 AM
not provided
Quality Gate: Passed